Auditor-Free Byzantine-Resilient Federated Learning via Functional Inner Products

DEEPAK GUPTA
2024PCS0024

A Dissertation Submitted to

Indian Institute of Technology, Jammu

In Partial Fulfillment of the Requirements for the Degree of

Master of Technology

in
Computer Science and Engineering

भारतीय प्रौद्योगिकी संस्थान जम्मू
INDIAN INSTITUTE OF TECHNOLOGY JAMMU

Department of Computer Science & Engineering

May 2026

DECLARATION

This thesis is submitted as a requirement for the award of the Master of Technology degree in the Department of Computer Science and Engineering, Indian Institute of Technology Jammu. I declare that this written submission represents my ideas in my own words, and where others' ideas or words have been included, I have adequately cited and referenced the original sources. The research embodied in this thesis has not been submitted in part or in full to any other University or Institute for the award of any other degree or diploma before. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated or falsified any idea/data/fact/source in my submission. Furthermore, I have not used any GenAI tools to generate the texts, except for grammatical checks and paraphrasing. I understand that any violation of the above will be a cause for disciplinary action by the Institute and can also evoke penal action from the sources that have not been properly cited, or from whom proper permission has not been obtained when required.

Name: Deepak Gupta
Roll No.: 2024PCS0024

APPROVAL SHEET

This thesis titled **Auditor-Free Byzantine-Resilient Federated Learning via Functional Inner Products** by **Deepak Gupta** is approved for the degree of **Master of Technology in Computer Science and Engineering** from IIT Jammu.

Date: 09/06/2026

Signature: _____

Dr. Avijit Dutta, TCG Crest
External Examiner

Signature: _____

Dr. Sumit Kumar Pandey, IIT Jammu
Internal Examiner

Signature: _____

Dr. Sudhakar Modem, IIT Jammu
Chairman

Signature: _____

Dr. Yamuna Prasad, IIT Jammu
Supervisor

To my parents,
who gave me the courage to dream big,
and to my teachers,
who showed me how to turn those dreams into reality.

Acknowledgement

I want to record my thanks to the people who helped this dissertation reach its present form. The work passed through many small phases (problem reading, protocol drafts, experiment runs, and long revision cycles), and each of those phases was made easier by someone around me.

My deepest thanks go to my supervisor, Dr. Yamuna Prasad, Head of the Department of Computer Science and Engineering at IIT Jammu. Three areas feed into this thesis: privacy-preserving federated learning, applied cryptography, and adversarial machine learning. I genuinely do not think I could have tied them together without his guidance. Whatever coherence the final work has is in large part his doing. He let me follow my own ideas, but he rarely let a loose assumption pass; more than once he sent me back to justify a step or to repair a proof I had thought was finished. I learned a lot from working with him, as much from his patience as from the standard he sets for the work.

I am grateful, too, to the M.Tech and Ph.D. scholars at IIT Jammu who shared the lab and many long evenings with me. Whiteboard arguments, bugs we traced down together, and unhurried research conversations all helped shape parts of this work, and the months of writing and revision were a good deal less solitary for them.

The Department of Computer Science and Engineering at IIT Jammu provided the academic atmosphere, coursework, and the compute that this study needed; I thank the department, and all the faculty who taught me during the first year, for that support.

Finally, my parents and my family have stood behind me through every phase of this thesis; to them I owe far more than a line in an acknowledgement.

Deepak Gupta

List of Publications

Conference Publications

1. Mahendra Kumar Gurve, Deepak Gupta, Nitin, and Yamuna Prasad, "Eliminating the Auditor: Functional Inner Product Based Byzantine-Resilient Federated Learning," accepted as a poster at the *ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys 2026)*, Saint-Malo, France, May 2026.

Abstract

Federated Learning, or FL, lets many clients jointly fit one model without sending their raw data anywhere. To tighten the privacy story, recent work wraps every client update in an additive homomorphic cipher, giving the family known as Privacy-Preserving Federated Learning (PPFL). The catch is that the same cipher which keeps a curious server out of the gradients also keeps the server from telling honest clients apart from bad ones. A Byzantine client can simply flip its labels and silently push the global model in a wrong direction, pulling accuracy down or planting a back-door on one chosen class.

Most defences here bring in an auditor: a trusted third party that decrypts cluster-level statistics of the incoming updates and screens out the bad ones with distributional models. They work, but at a price. The auditor is itself a new trust anchor, the very thing PPFL was meant to remove. Decrypting cluster summaries also leaks structural information about the gradients, and the cubic covariance step makes cost grow much faster than d. A defence that is at once fully encrypted, free of any extra trust anchor, and linear in d has so far been missing.

Functional Inner Product–Federated Learning, the framework developed in this thesis, is built around one idea: the verifier never needs to see a raw gradient. Working entirely under Paillier encryption, it computes one projection score per client: the Functional Inner Product (FIP) of that client's update against a fixed bootstrap reference direction. Once the FIP scalar is decrypted, the bootstrap audit is applied. The chapters that follow analyse this protocol along three axes: how much a scalar score actually reveals about the gradient, what region of update-space is still open to an attacker, and whether descent is preserved when the loss is L-smooth.

I built the framework end-to-end and ran it on MNIST and KDDCup99, with both IID and non-IID client splits, and with targeted and untargeted label-flipping attacks at malicious-client rates up to fifty percent. Across this grid, FIP-FL stays robust without any external auditor.

Contents

List of Figures

List of Tables

List of Abbreviations

ACC - Overall Accuracy

BFV - Brakerski–Fan–Vercauteren (homomorphic encryption scheme)

BGV - Brakerski–Gentry–Vaikuntanathan (homomorphic encryption scheme)

CKKS - Cheon–Kim–Kim–Song (approximate homomorphic encryption)

DCR - Decisional Composite Residuosity

DoS - Denial of Service

DP - Differential Privacy

FedAvg - Federated Averaging

FHE - Fully Homomorphic Encryption

FIP - Functional Inner Product

FIP-FL - Functional Inner Product Federated Learning (proposed framework)

FL - Federated Learning

FPR - False Positive Rate

GMM - Gaussian Mixture Model

HE - Homomorphic Encryption

IID - Independent and Identically Distributed

KDD - Knowledge Discovery in Databases

MNIST - Modified National Institute of Standards and Technology (database)

$O_{\mathbf{acc}}$ - Other-label Accuracy

PCA - Principal Component Analysis

PPFL - Privacy-Preserving Federated Learning

R2L - Remote-to-Local (KDDCup attack class)

SenSys - ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems

SGD - Stochastic Gradient Descent

SHA - Secure Hash Algorithm

SMPC - Secure Multiparty Computation

$T_{\mathbf{acc}}$ - Target-class Accuracy

TF-IDF - Term Frequency – Inverse Document Frequency

TKA - Trusted Key Authority

TPR - True Positive Rate

U2R - User-to-Root (KDDCup attack class)

1 Introduction

1.1 Background and Context

Federated Learning, FL for short, was put forward by McMahan and his coauthors [2] as a way of training a single global model without ever pulling the raw data into one place. The clients keep their samples on their own devices and send back only a gradient or a parameter update. That property is what makes the FL recipe useful for things like mobile keyboards, medical records, fraud-detection feeds, and anywhere else that the data is sensitive and naturally distributed.

But keeping the data local is, by itself, not really enough. The shared gradients can still leak training samples [3], leak membership, or leak distributional traits of the underlying records. The line of work that hardens this channel is called Privacy-Preserving Federated Learning, or PPFL. In privacy-preserving federated learning, three ideas keep coming up: differential privacy, secure aggregation, and homomorphic encryption. My work sits squarely in the third camp. By choosing additive homomorphic encryption, I let the server add together encrypted model updates while never seeing an individual gradient in plain text.

1.2 The Research Gap

That strong privacy shield has a flipside. Once every update reaches the server as ciphertext, the usual checks for bad behaviour stop working. A single dishonest client can flip labels, stretch its gradients, or inject a poison vector, and the server cannot tell. Well-known robust rules such as Krum [4], the coordinate-wise median, the trimmed mean [5], and Bulyan [6] all need plaintext inputs.

Detectors like Auror [7], FL-Trust [8], and PEFL [9] face the same roadblock because they rely on gradients they can read.

The most relevant baseline that does work over encrypted gradients is the auditor-based scheme of Yazdinejad et al. [1]. It is effective. But it depends on a trusted auditor, it decrypts cluster-level summaries to make decisions, it carries an $\mathcal{O}(d^3)$ covariance-inversion cost, and under non-IID partitions it exposes structural population information. The piece that is genuinely missing in the literature, then, is a defence that is at once auditor-free, robust, linear in d, and still usable when the federation is realistically heterogeneous.

1.3 Research Objectives and Contributions

What this thesis sets out to do is design, analyse, and evaluate FIP-FL, a Byzantine-resilient PPFL framework in which no auditor is involved. The verifier looks at each encrypted client update through a single functional inner product (FIP) against a fixed bootstrap reference direction; only one scalar per client per round ever needs to be decrypted. That scalar is then put through a bootstrap audit.

The main contributions of the thesis are the following.

1. **An auditor-free PPFL architecture.** FIP-FL drops the external auditor entirely and does its verification straight on the ciphertexts, using a scalar FIP score.

2. **A bootstrap audit.** The direction test discards the updates that point the wrong way, and after borderline restoration the audit rejects the lowest-scoring clients according to the attack-rate setting.

3. **Formal guarantees.** The thesis proves scalar-gradient privacy, gives the admissibility region left to an attacker, derives monotone descent under L-smoothness, and shows lower verifier complexity than the auditor-based baseline.

4. **Empirical validation.** FIP-FL is run on MNIST and on KDDCup99, under both IID and non-IID partitions, against targeted and untargeted label flipping, and with the malicious-client rate swept all the way to 50%.

5. **Engineering efficiency.** The verifier ends up running in $\mathcal{O}(Nd+N\log N)$, in place of an $\mathcal{O}(d^3)$ covariance step, and parallelising Phase II gives roughly a 13$\times$ wall-clock speed-up on 25 cores.

The research underlying this thesis has been accepted as a poster at the ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys 2026), Saint-Malo, France, May 2026.

2 Background and Motivation

2.1 Federated Learning and FedAvg

Take N clients. Client i holds private data $\mathcal{D}_i = \{(x_k^{(i)}, y_k^{(i)})\}_{k=1}^{n_i}$, and the total sample count is $n = \sum_i n_i$. The federated objective is then the size-weighted empirical risk

$$F(\mathbf{w}) = \sum_{i=1}^{N} \frac{n_i}{n} F_i(\mathbf{w}), \qquad F_i(\mathbf{w}) = \frac{1}{n_i} \sum_{k=1}^{n_i} \ell(f_{\mathbf{w}}(x_k^{(i)}), y_k^{(i)}). \tag{2.1}$$

The FedAvg algorithm of McMahan et al. [2] attacks this objective in the obvious way: send $\mathbf{w}^{(t)}$ down to every client, let each client run local SGD on its own shard, and average back what comes up. Writing the descent direction submitted by client i as $\Delta\boldsymbol{\theta}_i^{(t)} = \mathbf{w}^{(t)} - \mathbf{w}_i^{(t+1)}$, the server step boils down to

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \sum_{i=1}^{N} \frac{n_i}{n} \Delta\boldsymbol{\theta}_i^{(t)}. \tag{2.2}$$

So the server is left with a coordinate-wise weighted sum, and the raw samples themselves never have to move.

2.2 IID and Non-IID Federations

Many FedAvg studies pretend every client sees the same data distribution. Real deployments are messier. Phones, clinics, and sensors each carry their own label mix and feature quirks. I focus on label skew, meaning a client only sees part of the label set. Honest gradients then point in different directions, so any defence must separate natural drift from a real attack.

2.3 Privacy-Preserving Federated Learning

Raw gradients leak more than most people realise. A curious server can rebuild samples or test membership [3]. Three main defences exist: differential privacy, secure aggregation, and homomorphic encryption. Differential privacy adds noise but also hides useful signal. Secure aggregation protects each update yet cannot drop a bad one before the sum [10]. Homomorphic schemes keep everything encrypted while still allowing limited math. Paillier [11] handles addition and scaling cheaply; BFV and CKKS [12] support richer operations at a higher cost. I go with Paillier because my audit is simple: measure each encrypted update against a public reference vector through an inner product, all in linear time.

2.4 Poisoning Attack Taxonomy

I look at two label-flipping tricks. The untargeted version cycles every label and drags overall accuracy down. The targeted one maps a single source class to a target class and hurts that prediction. I leave out sign flips, scaling, model replacement, backdoors [14], and collusion, though my audit still checks the direction and size of every update to keep those threats in mind.

3 Literature Survey

3.1 Robust Aggregation Rules

Some defences swap the mean for sturdier estimators. Krum and Multi-Krum [4] pick updates close to many neighbours but need every pairwise distance and cost $\mathcal{O}(N^2 d)$. The coordinate-wise median and trimmed mean [5] cut that to $\mathcal{O}(Nd \log N)$ yet struggle when genuine data vary strongly.

3.2 Detection-Based Defences

Other methods try to catch bad clients first. Most still need plaintext. PEFL [9] works on ciphertext but slows down when attacks rise or data skew deepens [1]. FoolsGold [15] tracks long-term similarity, while ShieldFL [16] brings in a helper. Strong detection often means plaintext, a trusted partner, or extra bandwidth. My approach avoids all three by releasing just one scalar per client.

3.3 Auditor-Based PPFL Baseline

Yazdinejad and colleagues [1] add a trusted auditor who decrypts cluster summaries, fits a Gaussian mixture, and drops outliers with Mahalanobis distance before homomorphic averaging. They reach about 96.5 percent target-class accuracy on non-IID MNIST under a fifty-percent flip, but that success requires an extra trust anchor, $\mathcal{O}(d^3)$ covariance inversion, and leaking cluster stats.

3.4 Comparative Summary of Related Work

The comparison below summarises representative Byzantine-resilient and privacy-preserving federated learning defences that are closest to the setting considered in this thesis. Each entry highlights the method's core idea, main advantage, major limitation, and relevance to the proposed FIP-FL framework.

Work / defence: Krum / Multi-Krum [4]. Main idea: Selects the update that is closest to most other client updates using pairwise distances. **Advantage:** Provides Byzantine robustness in plaintext FL and is a standard robust aggregation baseline. **Limitation:** Requires access to plaintext gradients and incurs pairwise-distance computation over client updates; its distance-based assumption can also become less reliable when honest gradients are naturally diverse under non-IID data. **Relevance to FIP-FL:** Shows why classical robust aggregation cannot be directly used when gradients are encrypted.

Work / defence: Coordinate-wise Median / Trimmed Mean [5]. Main idea: Replaces the mean with coordinate-wise robust statistics by taking median values or trimming extreme coordinates. **Advantage:** Simpler and computationally cheaper than pairwise-distance-based robust rules. **Limitation:** Still requires plaintext coordinates and may remove useful honest variation under heterogeneous client data. **Relevance to FIP-FL:** Motivates the need for a ciphertext-compatible filtering rule.

Work / defence: FLTrust [8]. Main idea: Uses a trusted server-side reference dataset to compute a reference gradient and score client updates by their alignment with it. **Advantage:** Provides a strong reference-based trust signal against poisoning. **Limitation:** Requires a clean server-held dataset and generally assumes access to readable gradient information, which limits its fit for strict PPFL. **Relevance to FIP-FL:** FIP-FL also uses a reference direction, but computes only an encrypted functional inner product instead of exposing full gradients.

Work / defence: PEFL [9] / ShieldFL [16]. Main idea: Represents privacy-preserving poisoning-defence approaches that try to protect client updates while detecting malicious behaviour. **Advantage:** Moves closer to the PPFL setting than plaintext-only robust aggregation methods. **Limitation:** PEFL can become less efficient under stronger attacks or deeper data skew, while ShieldFL

introduces additional helper, trust, or protocol assumptions. **Relevance to FIP-FL:** Helps position FIP-FL as a lighter auditor-free encrypted-gradient defence.

Work / defence: Auditor-based PPFL [1]. Main idea: Uses additive homomorphic encryption with an internal auditor; malicious gradients are detected using Gaussian Mixture Models and Mahalanobis Distance. **Advantage:** Directly addresses encrypted gradients, targeted and untargeted attacks, and IID/non-IID settings. **Limitation:** Relies on a trusted auditing component, performs richer distributional auditing over gradient statistics, and requires covariance-based computations that increase cost with model dimension. **Relevance to FIP-FL:** This is the closest baseline. FIP-FL keeps the encrypted-gradient setting but replaces auditor-side distributional modelling with one scalar functional inner product per client.

From this comparison, the gap is clear. Classical robust aggregation methods offer Byzantine tolerance but require plaintext gradients. Detection-based methods improve poisoning resistance but often depend on readable gradients, trusted validation data, or extra helper entities. Auditor-based PPFL handles encrypted gradients, but it introduces a trusted auditing component and relies on richer distributional statistics for auditing. FIP-FL is designed to close this gap by keeping updates encrypted, removing the auditor, and reducing verification to a single functional inner product score per client.

3.5 Positioning of FIP-FL

My method, FIP-FL, keeps every update encrypted, calls on no auditor, swaps clustering for a simple alignment check, and stays linear in the model dimension d. The goal is straightforward: hold privacy tight while keeping Byzantine screening light.

4 Preliminaries

4.1 Notation

Scalars are lowercase italics (a, η), vectors are lowercase bold ($\mathbf{x}$, $\boldsymbol{\theta}$), and matrices are uppercase bold ($\mathbf{W}$, $\mathbf{M}$). For $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$,

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{k=1}^{d} a_k b_k, \qquad \|\mathbf{a}\|_2 = \sqrt{\langle \mathbf{a}, \mathbf{a} \rangle}. \tag{4.1}$$

A Paillier ciphertext is written $[\![m]\!] = \mathrm{Enc}_{pk}(m)$. The main FL symbols are N clients, model dimension d, round index t, local epochs E, batch size B, learning rate η, and Paillier security parameter λ.

4.2 The Paillier Cryptosystem

The additive homomorphic scheme my implementation rests on is Paillier [11]. Fix a security parameter λ. The Trusted Key Authority then draws two primes p and q, sets $n = pq$, takes the generator $g = n + 1$, and locks away the secret key $sk = (\ell, \alpha)$ with $\ell = \mathrm{lcm}(p - 1, q - 1)$ and $\alpha = \ell^{-1} \bmod n$. Throughout the thesis every experiment fixes $\lambda = 1024$.

For plaintext $m \in \mathbb{Z}_n$ and random $r \in \mathbb{Z}_n^*$,

$$\mathrm{Enc}_{pk}(m) = g^m r^n \pmod{n^2}, \tag{4.2}$$

and decryption is

$$\mathrm{Dec}_{sk}(c) = L(c^\ell \bmod n^2)\alpha \pmod{n}, \qquad L(x) = \frac{x - 1}{n}. \tag{4.3}$$

For plaintexts m_1, m_2 and known integer a,

$$\mathrm{Enc}_{pk}(m_1)\mathrm{Enc}_{pk}(m_2) \equiv \mathrm{Enc}_{pk}(m_1 + m_2) \pmod{n^2}, \tag{4.4}$$

$$\mathrm{Enc}_{pk}(m_1)^a \equiv \mathrm{Enc}_{pk}(am_1) \pmod{n^2}. \tag{4.5}$$

Between them, these two identities cover everything FIP-FL ever asks for: encrypted sums and encrypted linear functionals. The protocol never once multiplies two ciphertexts together.

Gradients are real-valued, so before encryption they are turned into fixed-point integers. For a scale s,

$$\mathrm{encode}(x) = \mathrm{round}(sx), \qquad \mathrm{decode}(z) = z/s. \tag{4.6}$$

Signed values live inside the Paillier plaintext ring through centered modular encoding: a negative $-a$ is stored as $n - a$ and read back from the interval $(-n/2, n/2]$. Negative plaintext scalars that turn up during homomorphic multiplication are dealt with by modular inverses. The experiments fix $s = 10{,}000$. One bookkeeping detail is worth flagging here: an encrypted FIP score multiplies an encoded update coordinate by an encoded reference coordinate, so once decrypted it has to be divided by s^2 to land back on the right scale.

4.3 Functional Inner Product

When the gradient vectors are flat real vectors, the Functional Inner Product used throughout this thesis is just the ordinary Euclidean dot product

$$\langle \mathbf{g}_i, \mathbf{g}_{\mathrm{ref}} \rangle = \sum_{k=1}^{d} g_i^{(k)} g_{\mathrm{ref}}^{(k)}. \tag{4.7}$$

The name deliberately echoes inner-product functional encryption [17], which likewise reveals only an inner product of an otherwise hidden vector; in FIP-FL, however, that functional is realised through Paillier homomorphic operations rather than a dedicated functional-encryption scheme. In plain terms, the FIP score measures how well a client's gradient update aligns with the trusted reference direction, nothing more than that. Feed (4.7) into the Paillier identities for plaintext-scalar multiplication and ciphertext addition, and the score can be

assembled entirely under encryption:

$$[\![\langle \mathbf{g}_i, \mathbf{g}_{\text{ref}} \rangle]\!] = \prod_{k=1}^{d} [\![g_i^{(k)}]\!]^{\text{encode}(g_{\text{ref}}^{(k)})} \pmod{n^2}. \tag{4.8}$$

This lone scalar is the only thing ever decrypted, and even that step belongs to the TKA. Not a single coordinate of $\mathbf{g}_i$ is exposed anywhere in the round.

4.4 System and Threat Model

The FIP-FL protocol has four roles. The clients submit encrypted gradients. The Aggregation Server takes whatever ciphertexts the audit has accepted and broadcasts the new model. The FIP Verifier is the role that actually computes the scalar FIP scores and runs the bootstrap audit. The Trusted Key Authority (TKA) generates the Paillier keys and is allowed to decrypt only those scalar scores and aggregates that the protocol explicitly asks it to. The Verifier may be co-located with the server in practice; in either case, no external auditor enters the trust model.

The adversary in this thesis controls up to $f < N/2$ Byzantine clients. Such an adversary is free to relabel its own data, hand in any syntactically valid ciphertext it likes, coordinate with other malicious clients, and listen in on every public broadcast. The empirical part instantiates this model with targeted and untargeted label flipping, at malicious rates $f/N \in \{10\%, 20\%, 30\%, 50\%\}$.

4.5 Security and Privacy Goals

The protocol is designed against four targets. *Privacy*: neither the server nor the Verifier should learn any coordinate, norm, or pairwise distance over the client gradients beyond the scalar score $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{g}_{\text{ref}}^{(t)} \rangle$. *Byzantine resilience*: an update is accepted only when it passes the bootstrap audit. *Convergence*: the average of the accepted updates has to satisfy the descent-alignment plus step-size pair stated in Chapter 5. *Efficiency*: the verifier should run in $\mathcal{O}(Nd + N \log N)$, and the aggregation step in $\mathcal{O}(|\mathcal{A}^{(t)}|d)$ for the accepted set $\mathcal{A}^{(t)}$.

5 Proposed Framework: FIP-FL

5.1 System Architecture

FIP-FL is built around four logical roles: the N clients, an Aggregation Server, a FIP Verifier, and a Trusted Key Authority (or TKA). In a single round, the clients put their Paillier ciphertexts on the wire, the Verifier turns each ciphertext into one alignment score, only the accepted ciphertexts make it to the aggregation server, and the TKA touches at most scalar scores or aggregate updates, never an individual plaintext gradient.

The system is organized so that each component has a clearly limited role. Clients perform local training and encrypt their updates. The Verifier evaluates these encrypted updates and filters suspicious ones. The server aggregates only the updates that pass the audit, while the TKA is responsible solely for the decryptions explicitly required by the protocol.

This separation is intentional. The Verifier can access encrypted client updates and the plaintext reference direction $\mathbf{R}^{(t)}$, but it never sees any individual gradient in plaintext form. Conversely, the TKA decrypts scalar FIP scores and aggregated updates, yet it has no access to the audit logic itself.

5.2 System Setup and Key Distribution

The setup phase is executed only once, before round 1. As described in Algorithm 1, the protocol keeps the three forms of state deliberately separate. The cryptographic state is maintained by the TKA. The model state begins with a clean global model stored on the server. The geometric state, namely the unit-norm reference direction, is initialized using a small public bootstrap dataset and then stored at the Verifier.

Algorithm 1 FIP-FL System Initialization and Reference Bootstrapping

Require: Number of clients N, model $f_{\boldsymbol{\theta}}$, public bootstrap set $\mathcal{D}_{\text{init}}$, security parameter λ, fixed-point scale s

Ensure: Registered clients, Paillier keys, initial global model $\boldsymbol{\theta}^{(0)}$, trusted reference direction $\mathbf{g}_{\text{ref}}^{(0)}$

Key generation and registration

$(pk, sk) \leftarrow \textsc{PaillierKeyGen}(1^{\lambda})$ at the Trusted Key Authority

TKA keeps sk sealed and releases only pk to the clients and the FIP-FL server

for $i \leftarrow 1$ **to** N **do**

    Register client C_i and assign anonymous tag TagID_i

    Send (pk, s) to client C_i

end for

Model and verifier initialisation

Initialise global model parameters $\boldsymbol{\theta}^{(0)} \leftarrow \mathbf{0}$

Broadcast $\boldsymbol{\theta}^{(0)}$ to all registered clients

Initialise encrypted-update table $\mathcal{U}^{(0)} \leftarrow \emptyset$

Reference bootstrapping

Compute the trusted bootstrap gradient on public data:

$\widetilde{\mathbf{g}}_{\text{ref}}^{(0)} \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f_{\boldsymbol{\theta}}; \mathcal{D}_{\text{init}}) \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{(0)}}$

Normalise the reference direction:

$\mathbf{g}_{\text{ref}}^{(0)} \leftarrow \widetilde{\mathbf{g}}_{\text{ref}}^{(0)} / \max\left(\left\| \widetilde{\mathbf{g}}_{\text{ref}}^{(0)} \right\|_2, \varepsilon \right)$

Store $\mathbf{g}_{\text{ref}}^{(0)}$ at the FIP Verifier for round 1

The public bootstrap set plays a role at exactly one point in the protocol: it gives us $\mathbf{g}_{\text{ref}}^{(0)}$ at the zero model. From round $t \geq 1$ onwards we just call this reference direction $\mathbf{R}^{(t)}$.

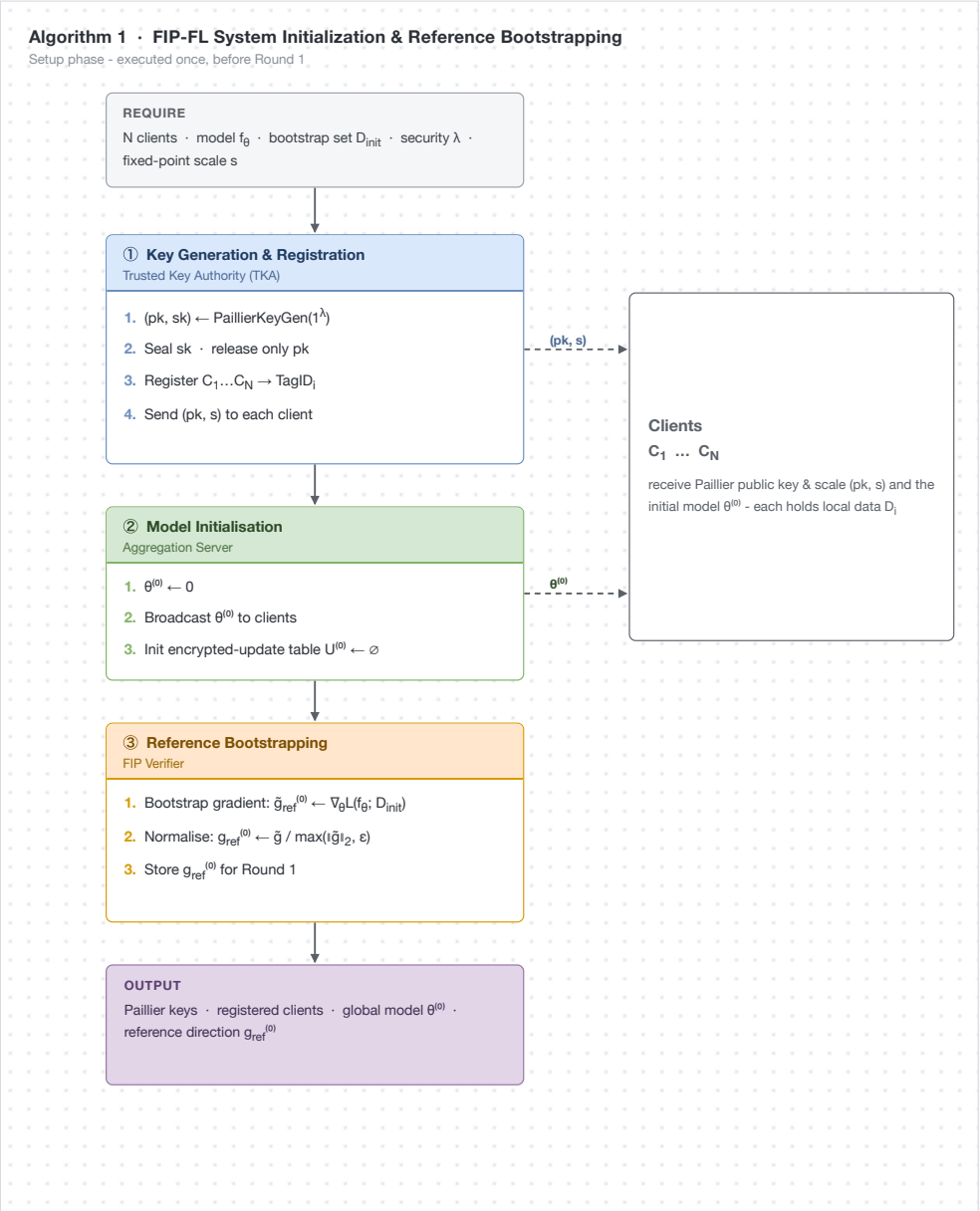


Figure 5.1: Setup and key-distribution flow of Algorithm 1.

5.3 Per-Round Protocol

After initialization, each round $t \geq 1$ follows the same core procedure described in the SenSys poster: encrypted gradient submission, FIP score computation, bootstrap-stage filtering, secure aggregation, and global model update.

Algorithm 2 Client-Side Local Training and Encrypted Gradient Submission

Require: Client C_i with data $\mathcal{D}_i$, round model $\boldsymbol{\theta}^{(t)}$, public key pk, scale s, epochs E, batch size B, learning rate η

Ensure: Encrypted submission $\mathcal{M}_i^{(t)}$

$\boldsymbol{\theta}_i \leftarrow \boldsymbol{\theta}^{(t)}$
$\mathbf{g}_i^{(t)} \leftarrow \mathbf{0}$, $m_i \leftarrow 0$
for $e \leftarrow 1$ **to** E **do**
  for all mini-batches $\mathcal{B} \subset \mathcal{D}_i$ of size B **do**
    $\mathbf{h} \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(f_{\boldsymbol{\theta}_i}; \mathcal{B})$
    $\mathbf{g}_i^{(t)} \leftarrow \mathbf{g}_i^{(t)} + \mathbf{h}$, $m_i \leftarrow m_i + 1$
    $\boldsymbol{\theta}_i \leftarrow \boldsymbol{\theta}_i - \eta \mathbf{h}$
  end for
end for
$\mathbf{g}_i^{(t)} \leftarrow \mathbf{g}_i^{(t)} / \max(m_i, 1)$
Flatten $\mathbf{g}_i^{(t)}$ into coordinates $\left(g_{i,1}^{(t)}, \ldots, g_{i,d}^{(t)}\right)$
for $k \leftarrow 1$ **to** d **do**
  $\widehat{g}_{i,k}^{(t)} \leftarrow \textsc{Encode}_s\left(g_{i,k}^{(t)}\right)$
  $[\![g_{i,k}^{(t)}]\!] \leftarrow \textsc{Enc}_{pk}\left(\widehat{g}_{i,k}^{(t)}\right)$
end for
$\mathcal{M}_i^{(t)} \leftarrow \left(\mathrm{TagID}_i, \{[\![g_{i,k}^{(t)}]\!]\}_{k=1}^{d}\right)$
Send $\mathcal{M}_i^{(t)}$ to the FIP Verifier

5.3.1 Phase I: Local Training and Encrypted Update Generation

In Phase I, each client runs E epochs of mini-batch SGD on its own shard, averages the gradients, flattens the weight and bias parameters into a single d-dimensional vector, encrypts each coordinate under Paillier, and ships the resulting ciphertext vector off to the Verifier. Algorithm 2 writes out the honest version of this; a malicious client tampers with its labels before this step ever begins, so its arithmetic on the device is the same.

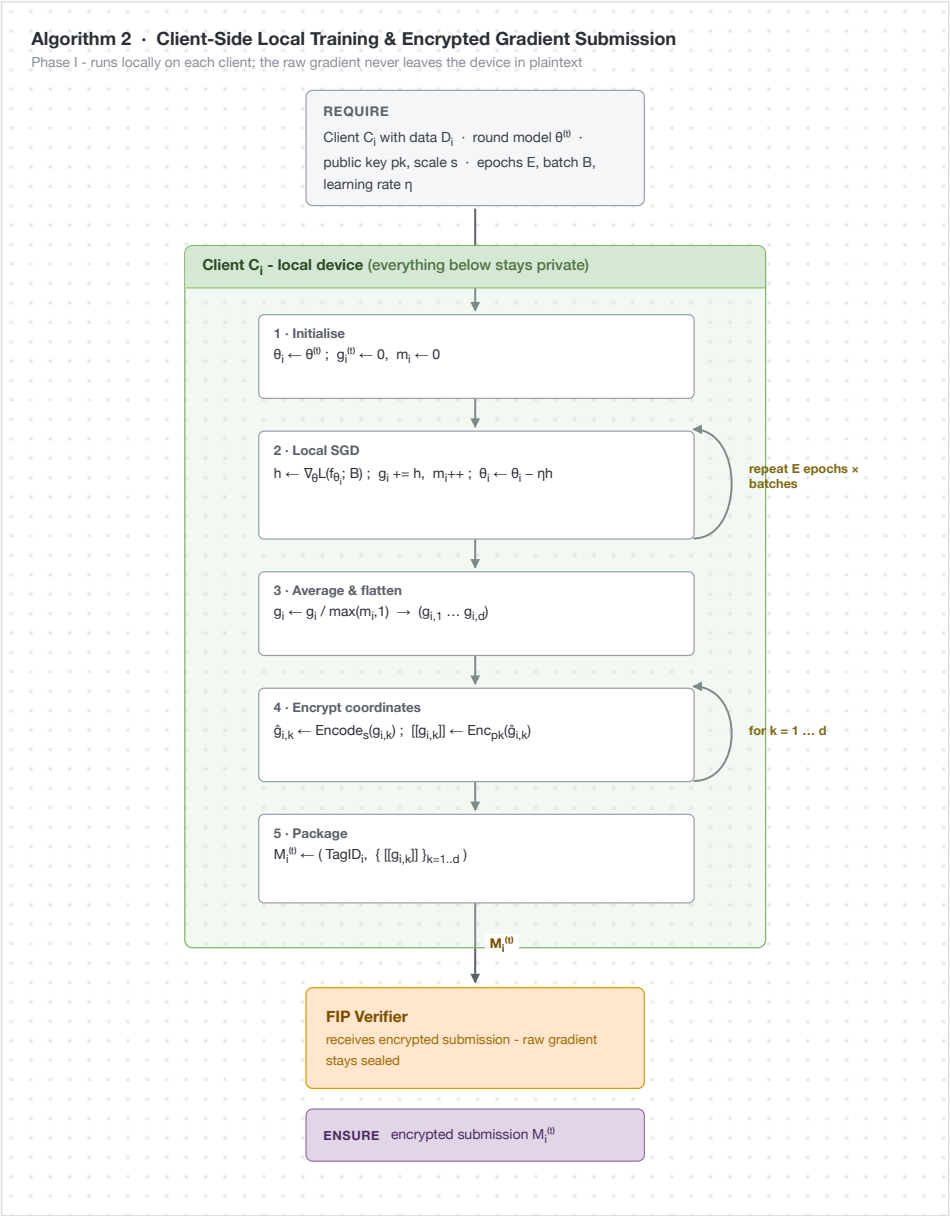


Figure 5.2: Phase I client pipeline of Algorithm 2.

5.3.2 Phase II: Homomorphic FIP Score Computation

In Phase II, the Verifier forms the projection of the client update onto $\mathbf{R}^{(t)}$ *while the update itself is still in ciphertext form.* The Paillier identities for plaintext-scalar multiplication and ciphertext addition take care of the inner product. After this is done, only the scalar $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{R}^{(t)} \rangle$ ever gets handed to the TKA for decryption; the individual coordinates of $\mathbf{g}_i^{(t)}$ stay sealed. Algorithm 3 writes this out step by step.

Algorithm 3 Homomorphic Functional Inner Product Score Computation

Require: Encrypted submissions $\mathcal{U}^{(t)}$, reference $\mathbf{R}^{(t)}$, public key pk, secret key sk held by TKA, scale s

Ensure: FIP score map $\mathcal{S}^{(t)}$

$\mathcal{S}^{(t)} \leftarrow \emptyset$

Encode reference coordinates $\widehat{R}_k^{(t)} \leftarrow \textsc{Encode}_s\left(R_k^{(t)}\right)$ for $k = 1, \ldots, d$

for all $\left(\mathrm{TagID}_i, \{[\![g_{i,k}^{(t)}]\!]\}_{k=1}^{d}\right) \in \mathcal{U}^{(t)}$ **do**

  $[\![\widehat{s}_i^{(t)}]\!] \leftarrow \textsc{Enc}_{pk}(0)$

  for $k \leftarrow 1$ **to** d **do**

    $[\![\widehat{s}_i^{(t)}]\!] \leftarrow [\![\widehat{s}_i^{(t)}]\!] \cdot \left([\![g_{i,k}^{(t)}]\!]\right)^{\widehat{R}_k^{(t)}} \bmod n^2$

  end for

  Send $[\![\widehat{s}_i^{(t)}]\!]$ to TKA for scalar decryption

  $\widehat{s}_i^{(t)} \leftarrow \textsc{Dec}_{sk}\left([\![\widehat{s}_i^{(t)}]\!]\right)$

  $s_i^{(t)} \leftarrow \textsc{Decode}_{s^2}\left(\widehat{s}_i^{(t)}\right)$

  $\mathcal{S}^{(t)}[\mathrm{TagID}_i] \leftarrow s_i^{(t)}$

end for

return $\mathcal{S}^{(t)}$

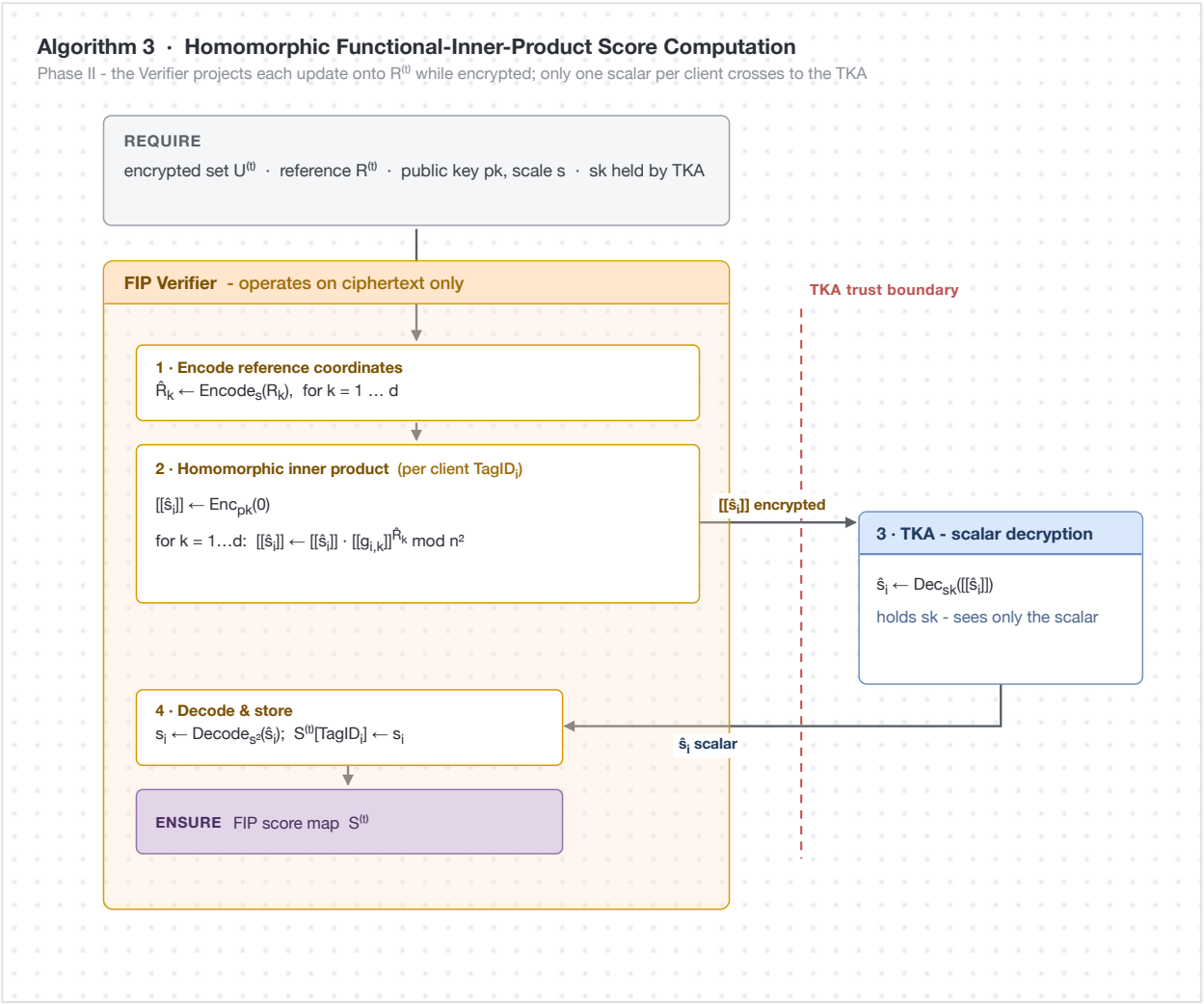


Figure 5.3: Phase II homomorphic scoring of Algorithm 3.

5.3.3 Phase III: Adaptive Bootstrap-Only Stage-1 Audit

The audit begins with a direction test on the bootstrap FIP scores. Clients whose scores exceed the direction threshold are accepted, while the remaining clients are placed in a rejected set. If this initial pass leaves too few accepted clients, borderline rejected clients are restored within a noise-aware band around the threshold.

The final step uses the attack-rate setting to keep only the lowest-scoring rejected clients. All other rejected clients are restored to the accepted set. The complete bootstrap-stage procedure is given in Algorithm 4.

Algorithm 4 Adaptive Bootstrap-Only Stage-1 Audit

Require: Bootstrap FIP scores $S^{(t)}$, direction threshold ϵ_{dir}, attack-rate setting ρ, noise scale σ_{noise}

Ensure: Accepted set $\mathcal{A}^{(t)}$, rejected set $\mathcal{R}^{(t)}$

$\mathcal{A}^{(t)} \leftarrow \emptyset$
$\mathcal{R}^{(t)} \leftarrow \emptyset$
for all $\left(\text{TagID}_i, s_i^{(t)}\right) \in S^{(t)}$ **do**
    if $s_i^{(t)} > \epsilon_{\text{dir}}$ **then**
        $\mathcal{A}^{(t)} \leftarrow \mathcal{A}^{(t)} \cup \{\text{TagID}_i\}$
    else
        $\mathcal{R}^{(t)} \leftarrow \mathcal{R}^{(t)} \cup \{\text{TagID}_i\}$
    end if
end for
$K_{\min} \leftarrow \lceil 0.5N \rceil$
$\delta_R \leftarrow \max(3 \cdot \sigma_{\text{noise}},\, 0.03)$
while $|\mathcal{A}^{(t)}| < K_{\min}$ **do**
    Select $C_j \in \mathcal{R}^{(t)}$ with the largest bootstrap score
    if $\left|s_j^{(t)} - \epsilon_{\text{dir}}\right| \leq \delta_R$ **then**
        $\mathcal{A}^{(t)} \leftarrow \mathcal{A}^{(t)} \cup \{C_j\}$
        $\mathcal{R}^{(t)} \leftarrow \mathcal{R}^{(t)} \setminus \{C_j\}$
    else
        break
    end if
end while
$B \leftarrow \text{round}(\rho N)$
Sort $\mathcal{R}^{(t)}$ in ascending order of bootstrap score
$\mathcal{R}_{\text{keep}}^{(t)} \leftarrow$ the B lowest-score rejects in $\mathcal{R}^{(t)}$
$\mathcal{A}^{(t)} \leftarrow \mathcal{A}^{(t)} \cup \left(\mathcal{R}^{(t)} \setminus \mathcal{R}_{\text{keep}}^{(t)}\right)$
$\mathcal{R}^{(t)} \leftarrow \mathcal{R}_{\text{keep}}^{(t)}$
return $\mathcal{A}^{(t)}, \mathcal{R}^{(t)}$

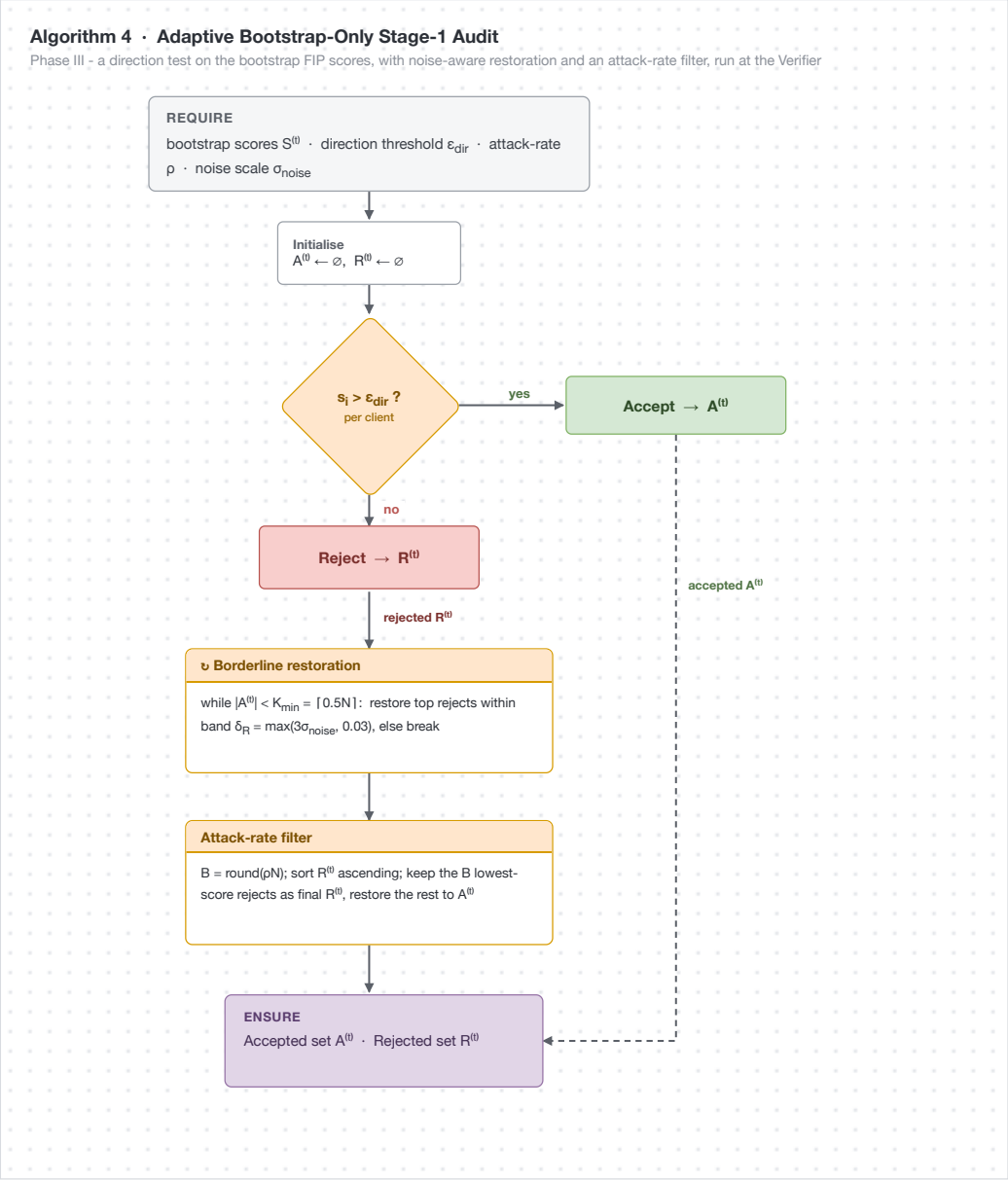


Figure 5.4: Phase III bootstrap-stage audit of Algorithm 4.

5.3.4 Phase IV: Secure Aggregation

Phase IV is short. The ciphertexts that the audit rejected are simply dropped; they are never decrypted at all. Whatever survives is folded into one pooled encrypted sum, and only that sum is later opened.

Algorithm 5 Secure Aggregation and Model Update

Require: Accepted encrypted set $\mathcal{A}^{(t)}$, model $\boldsymbol{\theta}^{(t)}$, public key pk, secret key sk, fixed-point scale s, learning rate η

Ensure: Updated model $\boldsymbol{\theta}^{(t+1)}$

if $\mathcal{A}^{(t)} = \emptyset$ **then**
  $\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)}$
  Broadcast $\boldsymbol{\theta}^{(t+1)}$
  return $\boldsymbol{\theta}^{(t+1)}$
end if
$m \leftarrow |\mathcal{A}^{(t)}|$
for $k \leftarrow 1$ **to** d **do**
  $[\![\widehat{G}_k^{(t)}]\!] \leftarrow \textsc{Enc}_{pk}(0)$
  for all $\left(\text{TagID}_i, \{[\![g_{i,\ell}^{(t)}]\!]\}_{\ell=1}^{d}\right) \in \mathcal{A}^{(t)}$ **do**
    $[\![\widehat{G}_k^{(t)}]\!] \leftarrow [\![\widehat{G}_k^{(t)}]\!] \cdot [\![g_{i,k}^{(t)}]\!] \bmod n^2$
  end for
  Send $[\![\widehat{G}_k^{(t)}]\!]$ to TKA for aggregate decryption
  $\widehat{G}_k^{(t)} \leftarrow \textsc{Dec}_{sk}\left([\![\widehat{G}_k^{(t)}]\!]\right)$
  $g_{\text{global},k}^{(t)} \leftarrow \textsc{Decode}_s\left(\widehat{G}_k^{(t)}\right)/m$
end for
$\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} - \eta \mathbf{g}_{\text{global}}^{(t)}$
Broadcast $\boldsymbol{\theta}^{(t+1)}$ ▷ Bootstrap reference remains fixed
return $\boldsymbol{\theta}^{(t+1)}$

5.3.5 Phase V: Model Update and Distribution

In Phase V, the server updates the global model using the averaged accepted gradient. The server decrypts only the aggregate update, never an individual client update. If all client updates are rejected in a given round, the current model is carried forward unchanged to the next round.

Algorithm 5 presents secure aggregation and model update together in a single procedure. The bootstrap reference remains fixed during this step.

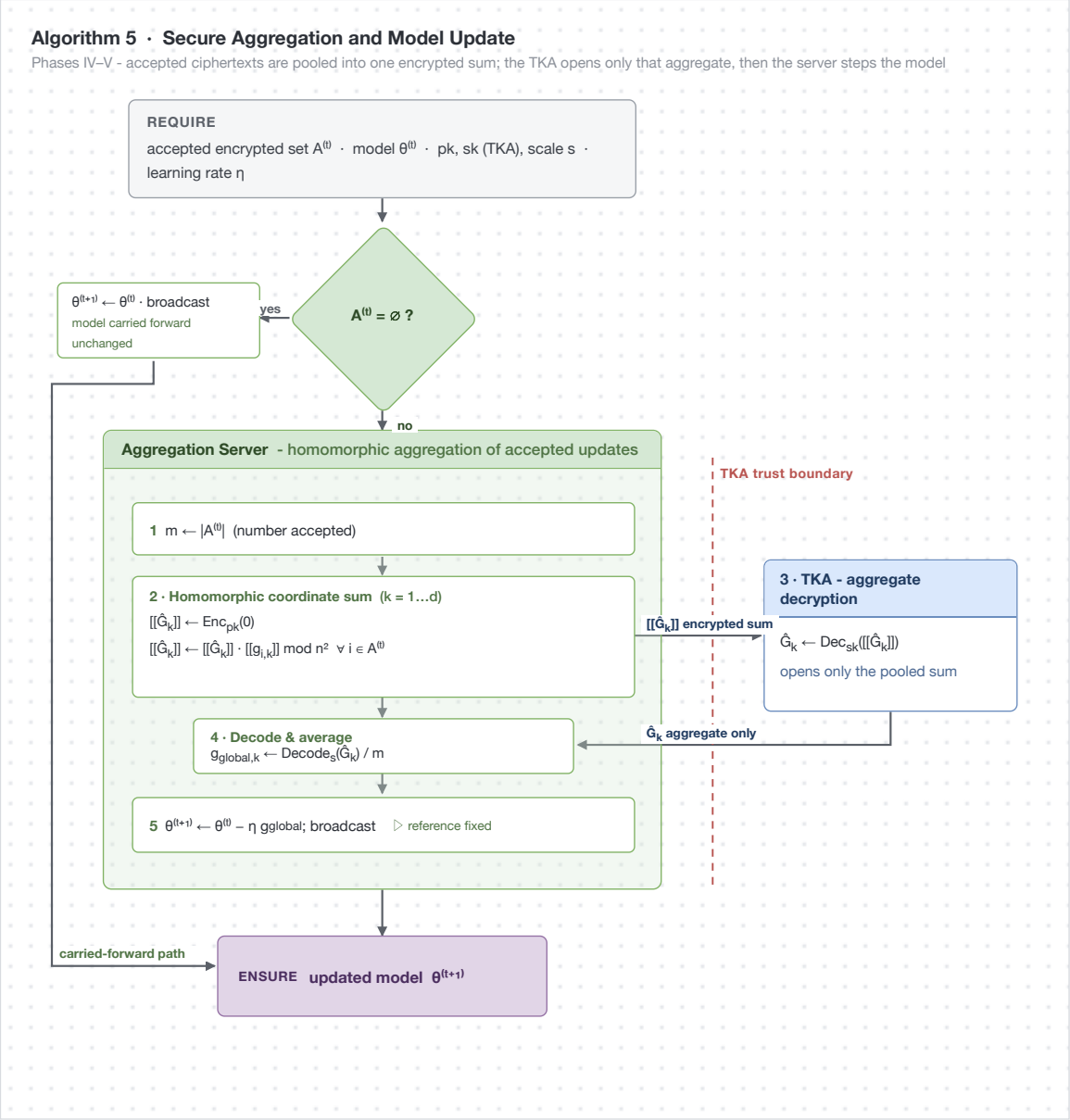


Figure 5.5: Phases IV–V secure aggregation and model update of Algorithm 5.

5.4 Theoretical Analysis

The three theorems collected below match, one to one, the four design targets stated in Section 4.5 (privacy, Byzantine resilience, and convergence; the efficiency target is a complexity statement and is read off directly from the algorithms above).

5.4.1 Gradient Privacy

Theorem 5.1 (Gradient Privacy)**.** *In every round t, the FIP Verifier reveals exactly one scalar per client, namely the projection $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{R}^{(t)} \rangle \in \mathbb{R}$. For a gradient of dimension $d \gg 1$, recovering $\mathbf{g}_i^{(t)}$ from $s_i^{(t)}$ is information-theoretically impossible: the linear system $\mathbf{R}^{(t)\top} \mathbf{g}_i^{(t)} = s_i^{(t)}$ is under-determined and admits a $(d-1)$-dimensional affine solution space. Equivalently, the mutual information satisfies $I(\mathbf{g}_i^{(t)}; s_i^{(t)}) \leq H(s_i^{(t)}) \ll H(\mathbf{g}_i^{(t)})$.*

Proof. The FIP Verifier's only operation on ciphertext $[\![\mathbf{g}_i^{(t)}]\!]$ is the homomorphic inner product of (4.8), which yields $[\![s_i^{(t)}]\!]$. Given $s_i^{(t)}$ and $\mathbf{R}^{(t)}$, the preimage of $\mathbf{g} \mapsto \langle \mathbf{g}, \mathbf{R}^{(t)} \rangle$ is a $(d-1)$-dimensional affine hyperplane. Gradients differing by a vector orthogonal to $\mathbf{R}^{(t)}$ are indistinguishable from the scalar alone, and the data-processing inequality gives $I(\mathbf{g}_i^{(t)}; s_i^{(t)}) \leq H(s_i^{(t)}) \ll H(\mathbf{g}_i^{(t)})$. □

5.4.2 Attack Resilience

Theorem 5.2 (Bootstrap Audit Rejection Rule)**.** *In every round t, the audit of Algorithm 4 rejects exactly the $B = \mathrm{round}(\rho N)$ lowest-scoring clients among those whose bootstrap FIP scores remain below the direction threshold ϵ_{dir} after borderline restoration; every other client is accepted. In particular, a direction-reversing update of the form $\mathbf{g}_{adv} \approx -\mathbf{g}_{honest}$ produces a score below ϵ_{dir} and, lying among the lowest scores, is rejected, while the audit process remains independent of any external auditor.*

Proof. The claim follows directly from the steps of Algorithm 4. The direction test places every client with $s_i^{(t)} > \epsilon_{\mathrm{dir}}$ in the accepted set $\mathcal{A}^{(t)}$ and every other client in the rejected set $\mathcal{R}^{(t)}$. If $|\mathcal{A}^{(t)}| < K_{\min}$, borderline clients whose scores lie within the noise-aware band δ_R of ϵ_{dir} are restored to $\mathcal{A}^{(t)}$ in descending order of score. The algorithm then sorts $\mathcal{R}^{(t)}$ in ascending order of bootstrap score, keeps only the $B = \mathrm{round}(\rho N)$ lowest-scoring clients in the rejected set, and restores all remaining clients to $\mathcal{A}^{(t)}$. A direction-reversing update scores below ϵ_{dir}, falls outside the restoration band, and sits among the lowest-scoring entries of the sorted rejected set, so it is retained in $\mathcal{R}^{(t)}$ and never enters the aggregate. □

5.4.3 Convergence Guarantee

Theorem 5.3 (Descent under L-Smoothness)**.** *Let $F(\mathbf{w})$ be L-smooth, and let $\Delta^{(t)}_{global}$ denote the mean of the accepted updates in round t. If the accepted aggregate is aligned with the descent direction of the true gradient, that is,*

$$\langle \nabla F(\mathbf{w}^{(t)}),\, \Delta^{(t)}_{global} \rangle > 0, \tag{5.1}$$

then any learning rate

$$0 < \eta < \frac{2\langle \nabla F(\mathbf{w}^{(t)}),\, \Delta^{(t)}_{global} \rangle}{L \|\Delta^{(t)}_{global}\|_2^2} \tag{5.2}$$

gives a strict one-round loss decrease:

$$F(\mathbf{w}^{(t+1)}) < F(\mathbf{w}^{(t)}). \tag{5.3}$$

Consequently, if this alignment holds in expectation over accepted rounds, the expected global loss decreases monotonically.

Proof. Starting from the L-smoothness descent lemma we get

$$F(\mathbf{w}^{(t)} - \eta \Delta^{(t)}_{\text{global}}) \leq F(\mathbf{w}^{(t)}) - \eta \langle \nabla F(\mathbf{w}^{(t)}), \Delta^{(t)}_{\text{global}} \rangle + \frac{L\eta^2}{2} \|\Delta^{(t)}_{\text{global}}\|_2^2. \tag{5.4}$$

The key idea is that the alignment assumption makes the linear term strictly negative, while the bound on η ensures that the quadratic term does not dominate it. Under this step-size condition, the right-hand side remains strictly smaller than $F(\mathbf{w}^{(t)})$.

In practice, the FIP-FL filter is designed to make this assumption reasonable. Only updates passing the bootstrap audit are included in $\Delta^{(t)}_{\text{global}}$. Importantly, the theorem does not require the reference direction to equal the true gradient; it states the average descent condition that the accepted aggregate must satisfy. □

6 Experimental Setup

This chapter pins down the empirical setting in which FIP-FL is run: which datasets we use, how the data is split among the clients, what attack the adversary mounts, which numbers we look at, and the implementation and hardware stack that the experiments actually live on.

6.1 Datasets

The evaluation grid runs on two benchmarks, one image task and one intrusion-detection task. The high-level statistics are listed in Table 6.1.

Table 6.1: Datasets used in the evaluation of FIP-FL.

Dataset	**Modality**	**Classes**	**Features**	**Samples**
MNIST	Image	10	784	60,000/10,000
KDDCup99	Tabular	5	41	$\approx$ 494K (10%)

MNIST. The MNIST corpus has 60,000 training and 10,000 test grayscale images at 28×28, labelled over the digits $\{0, 1, \ldots, 9\}$. Every image is flattened to a 784-dimensional vector and rescaled into the unit interval. The same dataset is the headline benchmark in Yazdinejad et al. [1], which makes it the direct comparison point against the auditor-based baseline.

KDDCup99. KDDCup99 is the canonical network-intrusion benchmark. The full archive contains around 4.9 million labelled connection records; the experiments here use the standard 10% subset, which is roughly 494,000 records. Each record exposes 41 features mixing traffic counters and protocol flags, and

the labels are collapsed into five classes, NORMAL, DOS, PROBE, R2L and U2R. Categorical fields are one-hot encoded and every feature is standardised. The class distribution is strongly imbalanced, which is exactly what makes this dataset a useful stress test: it forces the audit to behave even when the priors are far from uniform.

Model Architecture. For both datasets, a single-layer softmax logistic regression classifier is used. This choice is consistent with the model used in the SenSys 2026 poster as well as the auditor-based baseline. The total number of parameters is $d = (\text{input dim}) \cdot C + C$, where the final term corresponds to the bias.

A compact model is selected for two main reasons. First, Paillier encryption with $\lambda = 1024$ bits is still practical when the parameter dimension remains in the range of a few thousand. Second, the convexity and L-smoothness assumptions required in Theorem 5.3 are naturally satisfied by softmax logistic regression. In contrast, for deep neural networks these assumptions would hold only approximately.

6.2 Data Partitioning

Each dataset is sliced into $N = 10$ disjoint client shards, and I run the experiments under two partition regimes. In the IID regime the training set is shuffled once and dealt out into equal-sized shards, so each client ends up with roughly $|\mathcal{D}_{\text{train}}|/N$ samples and the expected label histogram on every shard matches the global one.

For the non-IID regime I use label skew. With a C-class task, client i gets a primary label set $\mathcal{L}_i \subset \{0, 1, \ldots, C-1\}$ of size $\lceil C/2 \rceil$, and the overlap between successive clients is $\lfloor C/4 \rfloor$. The client data are assigned using an 80/20 rule. For each client, 80% of the data comes from its primary labels, while the remaining 20% is sampled from the full training set. This partitioning strategy ensures that each client still retains a small number of samples from every class. The setup follows the evaluation protocol used in SenSys 2026 and is also similar to the non-IID configuration considered by Yazdinejad et al. [1].

At the same time, the partition is deliberately challenging because even

honest clients may produce different descent directions. As a result, the audit mechanism must tolerate normal data heterogeneity while still being able to identify malicious updates reliably.

6.3 Attack Models

Two pre-training label-flipping attacks make up the empirical sweep. Neither needs the adversary to know anything about Paillier or about the FIP-FL audit logic; both are plain data-side manipulations.

Untargeted label flipping. A malicious client relabels each local sample by a fixed cyclic offset,

$$y_k^{(i,\,\mathrm{mal})} \;=\; (y_k^{(i)} + \kappa) \bmod C. \tag{6.1}$$

On MNIST I take $\kappa = 5$, which gives a no-fixed-point permutation of the ten digits, and on KDDCup99 I take $\kappa = 2$ because the task only has five classes. The aim of this attack is to drag overall accuracy down by pulling each malicious client's local gradient toward a deliberately wrong classification objective.

Targeted label flipping. A malicious client touches only a single source class c_s and remaps it to a chosen target class c_t,

$$y_k^{(i,\,\mathrm{mal})} \;=\; \begin{cases} c_t & \text{if } y_k^{(i)} = c_s, \\ y_k^{(i)} & \text{otherwise.} \end{cases} \tag{6.2}$$

On MNIST the chosen source–target pair is $(c_s, c_t) = (0, 7)$, and on KDDCup99 I use the analogous pair drawn from the two classes that dominate in the skew regime. The point of this attack is to break one semantic class while leaving overall accuracy looking reasonable, so the relevant diagnostic metric is the target-class accuracy rather than the overall one.

Malicious-client rates. For every combination of dataset, partition and attack type, the malicious-client fraction is swept over $f/N \in \{10\%, 20\%, 30\%, 50\%\}$. The 50% row is the most demanding stress test, sitting right at the boundary that majority-based aggregation can in principle survive.

6.4 Evaluation Metrics

Five metrics are used in the experiments. Three of them evaluate model utility on held-out test data, while the remaining two measure how effectively the verifier distinguishes malicious clients from honest ones.

Target-class accuracy ($T_{\mathbf{acc}}$). Target-class accuracy evaluates performance only on test samples belonging to the targeted source class c_s:

$$T_{\text{acc}} = \frac{|\{(x,y) \in \mathcal{D}_{\text{test}} : y = c_s,\ \hat{f}(x) = c_s\}|}{|\{(x,y) \in \mathcal{D}_{\text{test}} : y = c_s\}|}. \tag{6.3}$$

A successful targeted attack reduces this value, whereas an effective defence keeps it close to the clean baseline. For untargeted attacks, class 0 is taken as a fixed reference class so that results remain comparable across rows.

Other-label accuracy ($O_{\mathbf{acc}}$). Other-label accuracy measures performance on all test samples whose labels are not c_s:

$$O_{\text{acc}} = \frac{|\{(x,y) \in \mathcal{D}_{\text{test}} : y \neq c_s,\ \hat{f}(x) = y\}|}{|\{(x,y) \in \mathcal{D}_{\text{test}} : y \neq c_s\}|}. \tag{6.4}$$

Together, target-class accuracy (T_{acc}) and other-label accuracy (O_{acc}) show whether the attack mainly affects the target class or also damages the rest of the task.

Overall accuracy (ACC). Overall accuracy (ACC) is the standard test accuracy over all classes:

$$\text{ACC} = \frac{|\{(x,y) \in \mathcal{D}_{\text{test}} : \hat{f}(x) = y\}|}{|\mathcal{D}_{\text{test}}|}. \tag{6.5}$$

This is the main utility metric for untargeted attack experiments.

True positive rate (TPR). TPR is the fraction of malicious clients rejected by the FIP-FL audit:

$$\text{TPR} = \frac{|\{i : \text{client } i \text{ is malicious and rejected}\}|}{|\{i : \text{client } i \text{ is malicious}\}|}. \tag{6.6}$$

A higher TPR means that fewer malicious clients pass through the verifier.

False positive rate (FPR). This is the fraction of honest clients that are wrongly rejected:

$$\text{FPR} = \frac{|\{i : \text{client } i \text{ is honest and rejected}\}|}{|\{i : \text{client } i \text{ is honest}\}|}. \tag{6.7}$$

This metric captures the cost of filtering honest participants. All five metrics are averaged over three independent random seeds. Standard deviation is reported only when it is greater than 0.5 percentage points.

6.5 Implementation and Hardware

The implementation is written in Python 3. `NumPy` is used for linear algebra, `scikit-learn` provides the dataset loaders and feature pipelines, and Paillier encryption is implemented using the `phe` library with a 1024-bit modulus. The FIP Verifier and Aggregation Server run as separate Python processes and communicate through in-memory queues. The clients are simulated as worker processes. Both encryption and the homomorphic inner-product computation are parallelised using `multiprocessing`.

The wall-clock results in Chapter 8 were measured on a Linux workstation with a 25-core CPU and 128 GB of RAM.

7 Results and Analysis

This chapter goes through the empirical results of FIP-FL on the grid defined in Chapter 6. Each table reports the relevant round-300 numbers, ACC, T_{acc}, O_{acc}, TPR, and FPR. Accuracies are quoted as percentages, and TPR and FPR are quoted as proportions in $[0, 1]$.

7.1 Non-IID Partition, Untargeted Attacks

Of the four non-IID rows, the untargeted one is in some ways the most demanding. The label skew has already pushed the honest-client gradients away from each other, and on top of that the malicious clients are sending in label-flipped gradients that genuinely point away from the descent direction. Overall accuracy is the right utility number to read here, with TPR sitting alongside it as the cost-of-filtering pair.

7.1.1 MNIST

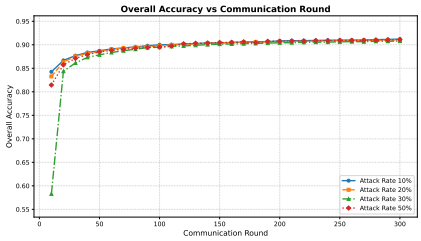
Table 7.1: MNIST, non-IID partition, untargeted label-flipping attack.

AR	**ACC (%)**	$T_{\mathbf{acc}}$ **(%)**	$O_{\mathbf{acc}}$ **(%)**	**TPR**
10%	91.22	97.60	90.18	1.00
20%	91.02	97.87	89.94	1.00
30%	90.78	97.33	89.31	1.00
50%	91.10	97.51	89.91	1.00

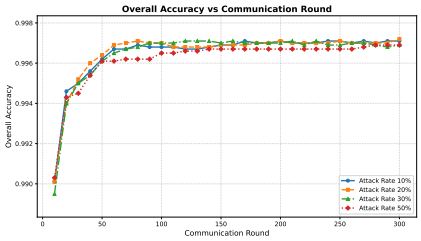
7.1.2 KDDCup99

Table 7.2: KDDCup99, non-IID partition (Dirichlet $\alpha = 0.5$), untargeted label-flipping attack.

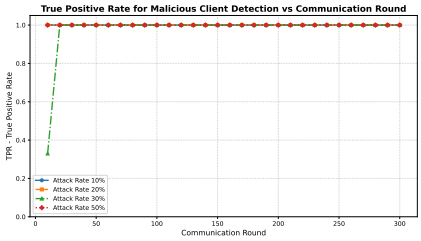
AR	ACC (%)	$T_{\mathbf{acc}}$ (%)	$O_{\mathbf{acc}}$ (%)	TPR
10%	99.71	99.50	84.62	1.00
20%	99.72	99.55	83.65	1.00
30%	99.69	99.45	85.58	1.00
50%	99.69	99.70	77.88	1.00



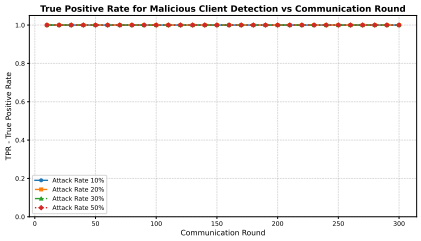
(a) MNIST ACC trajectory



(b) KDDCup99 ACC trajectory



(c) MNIST malicious-client TPR



(d) KDDCup99 malicious-client TPR

Figure 7.1: Non-IID untargeted attack dynamics over 300 communication rounds. Accuracy converges stably for both datasets, while the bootstrap audit reaches perfect malicious-client detection by the final round.

Observations. Across both datasets, FIP-FL ends up rejecting every single malicious update and the utility number stays close to its undefended-clean value even under the non-IID untargeted load. On MNIST the ACC sits between 90.78% and 91.22%, and false rejection only kicks in once the attack rate hits 50%. On KDDCup99 the ACC remains above 99.6% throughout the sweep, even at the point where the FPR begins to rise under the heaviest non-IID setting.

7.2 Non-IID Partition, Targeted Attacks

A targeted label-flipping attack is a more delicate kind of poisoning than the untargeted case. It tries to break one specific source class while letting the global accuracy stay believable, so reading ACC alone would actively hide the damage. The number that matters here is T_{acc}.

7.2.1 MNIST

Table 7.3: MNIST, non-IID, targeted label-flipping attack ($c_s = 0, c_t = 7$); SenSys 2026 headline targeted-accuracy cell, with the auditor-based value from [1].

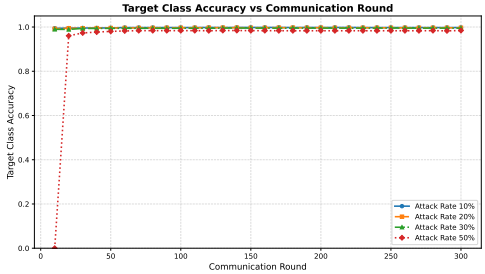
AR	**Undefended (%)**	**Auditor-based (%)**	**FIP-FL (%)**
50%	≈ 7.00	96.50	97.69

7.2.2 KDDCup99

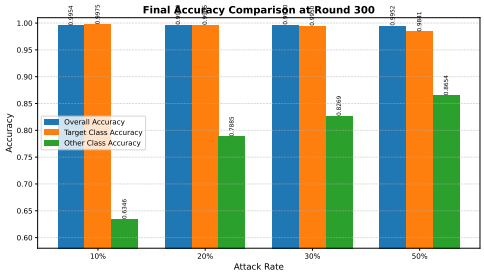
Table 7.4: KDDCup99, non-IID partition (2 shards/client), targeted label-flipping attack.

AR	**ACC (%)**	$T_{\textbf{acc}}$ **(%)**	$O_{\textbf{acc}}$ **(%)**	**TPR**	**FPR**
10%	99.54	99.75	63.46	1.00	0.00
20%	99.69	99.65	78.85	1.00	0.00
30%	99.70	99.50	82.69	1.00	0.00
50%	99.52	98.41	86.54	1.00	0.00

Observations. The MNIST row is the headline cell from the accepted SenSys 2026 poster. With 50% of the clients turning malicious, FIP-FL pushes T_{acc} from roughly 7% in the undefended run to 97.69%, compared with the 96.5% reported for the auditor-based scheme of Yazdinejad et al. [1]. KDDCup99 follows the same detection pattern across the full attack-rate sweep, with no false positives anywhere in the table and $T_{\text{acc}} = 98.41\%$ at the 50% row.



(a) Target-class accuracy trajectory



(b) Round-300 accuracy comparison

Figure 7.2: KDDCup99 non-IID targeted attack results. The target-class trajectory remains high through training, and the round-300 comparison shows that overall and target-class accuracy stay stable even as the malicious-client fraction increases.

7.3 IID Partition, Untargeted Attacks

The IID rows are included mostly as a sanity-check baseline. With IID partitions the honest clients are statistically very close to one another, which means the distribution of FIP scores in any given round is comparatively tight, and detection should be the easiest setting we will see.

7.3.1 MNIST

Table 7.5: MNIST, IID partition, untargeted label-flipping attack.

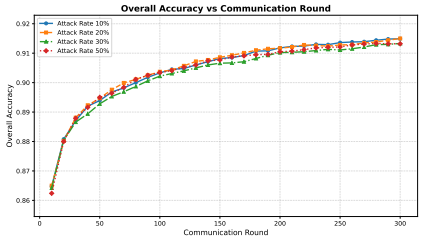
AR	**ACC (%)**	$T_{\mathbf{acc}}$ **(%)**	$O_{\mathbf{acc}}$ **(%)**	**TPR**	**FPR**
10%	91.49	97.51	90.40	1.00	0.00
20%	91.50	97.51	90.39	1.00	0.00
30%	91.33	97.42	90.28	1.00	0.00
50%	91.32	97.42	90.22	1.00	0.00

7.3.2 KDDCup99

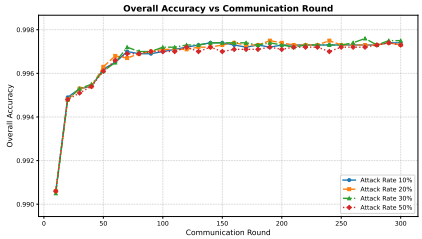
Table 7.6: KDDCup99, IID partition, untargeted label-flipping attack.

AR	ACC (%)	$T_{\mathbf{acc}}$ (%)	$O_{\mathbf{acc}}$ (%)	TPR	FPR
10%	99.74	99.60	85.58	1.00	0.00
20%	99.73	99.55	85.58	1.00	0.00
30%	99.75	99.60	85.58	1.00	0.00
50%	99.73	99.55	85.58	1.00	0.00

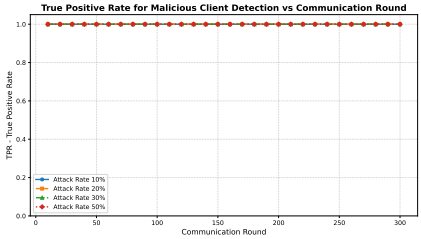
Observations. In the IID untargeted setting, detection is essentially exact on both datasets, every row reports TPR = 1.00 together with FPR = 0.00. The MNIST ACC moves by less than 0.2 percentage points across the whole attack-rate sweep, and on KDDCup99 the ACC, T_{acc}, and O_{acc} values together vary by at most 0.05 percentage points.



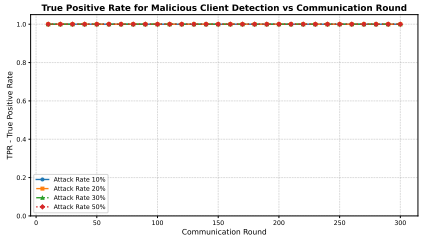
(a) MNIST ACC trajectory



(b) KDDCup99 ACC trajectory



(c) MNIST malicious-client TPR



(d) KDDCup99 malicious-client TPR

Figure 7.3: IID untargeted attack dynamics over 300 communication rounds. Both datasets show stable convergence, and the audit maintains perfect malicious-client detection without false positives at the reported final round.

7.4 IID Partition, Targeted Attacks

7.4.1 MNIST

Table 7.7: MNIST, IID partition, targeted label-flipping attack ($c_s = 0, c_t = 7$).

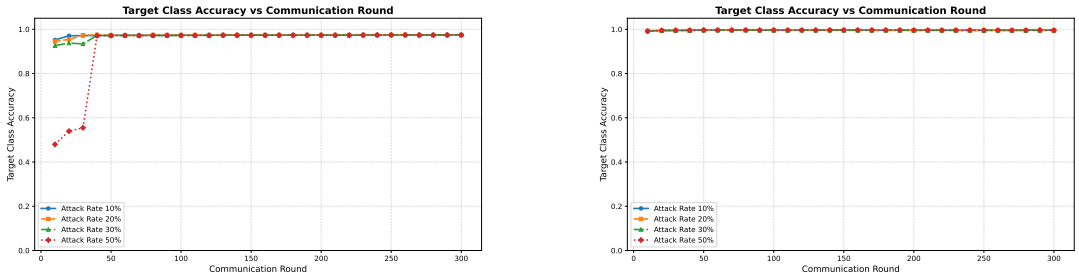
AR	**ACC (%)**	$T_{\mathbf{acc}}$ **(%)**	$O_{\mathbf{acc}}$ **(%)**	**TPR**	**FPR**
10%	91.62	97.51	90.57	1.00	0.00
20%	91.42	97.42	90.35	1.00	0.00
30%	91.32	97.42	90.22	1.00	0.00
50%	91.43	97.42	90.35	1.00	0.00

7.4.2 KDDCup99

Table 7.8: KDDCup99, IID partition, targeted label-flipping attack.

AR	**ACC (%)**	$T_{\mathbf{acc}}$ **(%)**	$O_{\mathbf{acc}}$ **(%)**	**TPR**	**FPR**
10%	99.73	99.60	84.62	1.00	0.00
20%	99.73	99.55	85.58	1.00	0.00
30%	99.72	99.50	85.58	1.00	0.00
50%	99.73	99.55	85.58	1.00	0.00

Observations. The IID targeted case lines up cleanly with the IID untargeted one. Detection stays perfect, false rejections never happen, and the target-class accuracy stays flat across all attack rates. MNIST holds $T_{\text{acc}} \geq 97.42\%$ across the table, and KDDCup99 keeps ACC in the 99.72%–99.73% band with $T_{\text{acc}} \geq 99.50\%$ on every row.



(a) MNIST target-class accuracy

(b) KDDCup99 target-class accuracy

Figure 7.4: IID targeted attack target-class accuracy over 300 communication rounds. In both datasets, the target class recovers to a stable high-accuracy regime by the final round.

7.5 Aggregate Findings Across the Grid

Stepping back from the individual tables, the grid as a whole supports three conclusions. First, every completed row reports TPR = 1.00, including the 50% malicious-client cases on both datasets. Second, false rejection only shows up under the heaviest non-IID untargeted load, exactly the regime where the honest gradients are already pulled apart by label skew, and even there ACC remains numerically stable. Third, targeted attacks are caught just as reliably as untargeted ones, which suggests that a one-dimensional FIP score against the trusted reference is genuinely sufficient for the poisoning regimes evaluated here. Chapter 8 puts these numbers next to the auditor-based baseline.

8 Comparative Analysis

This chapter places FIP-FL alongside the Byzantine-resilient FL defences that were already covered in Chapter 3, with the auditor-based baseline of Yazdinejad et al. [1] taking centre stage. The comparison runs over six axes, privacy leakage, computation, communication, robustness under non-IID skew, engineering cost, and the canonical MNIST non-IID targeted benchmark.

8.1 Privacy

The auditor-based scheme has to decrypt cluster-level statistics every round: a projected Gaussian-mixture model along with the Mahalanobis-distance statistics that the filter uses. Even though no individual gradient is ever reconstructed, the cluster means and covariances themselves still leak structural information about the client population. In a non-IID deployment, those statistics can correlate with demographics, hospital specialisation, keyboard language, or any number of other sensitive client traits.

FIP-FL is much more spare. The only thing decrypted per client per round is the scalar FIP score $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{R}^{(t)} \rangle$. There are no cluster means, no covariance matrices, no projection coefficients, no pairwise distances. The privacy improvement is categorical rather than incremental: the verifier sees a single alignment number, and the model update is still derived only from an aggregate.

8.2 Computational Complexity

Table 8.1: Per-round computational cost. N: clients; $|\mathcal{A}|$: accepted clients; d: model dimension; B: batch size; E: local epochs.

Entity	**Auditor-based [1]**	**FIP-FL**
Client	$\mathcal{O}(dBE)$	$\mathcal{O}(dBE)$
Verifier / Auditor	$\mathcal{O}(NKD + Nd\log d) + \mathcal{O}(d^3)$	$\mathcal{O}(Nd + N\log N)$
Aggregation Server	$\mathcal{O}(dN\log N)$	$\mathcal{O}(\lvert\mathcal{A}\rvert d)$

8.2.1 Time-Complexity Interpretation

The computational comparison in Table 8.1 shows that the main saving of FIP-FL appears at the verifier stage. The client-side cost remains unchanged because both schemes require each client to perform ordinary local SGD on its own data. The difference lies in how the submitted updates are verified and aggregated after local training.

Client-side training. The auditor-based PPFL scheme and FIP-FL both have client-side complexity $\mathcal{O}(dBE)$. Both schemes have the same local learning cost because each client trains on mini-batches for E local epochs.

Verification basis. The auditor-based PPFL scheme verifies updates through distributional modelling of submitted gradient statistics, while FIP-FL verifies updates through scalar alignment with the bootstrap reference direction. FIP-FL reduces verification from high-dimensional distributional analysis to one scalar score per client.

Verifier / Auditor operations. The auditor-based scheme uses projection, Gaussian-mixture modelling, Mahalanobis-distance scoring, and covariance-related computation. FIP-FL uses homomorphic functional inner product computation and bootstrap score filtering. The auditor-based scheme performs richer statistical modelling, whereas FIP-FL performs one encrypted inner product per client.

Verifier / Auditor complexity. The auditor-based verifier has complexity

$\mathcal{O}(NKD + Nd \log d) + \mathcal{O}(d^3)$, while the FIP-FL verifier has complexity $\mathcal{O}(Nd + N \log N)$. Under the asymptotic model used in this thesis, the additional $\mathcal{O}(d^3)$ term accounts for covariance-inversion style computation in the auditor-side distributional model.

Aggregation cost. The auditor-based aggregation cost is $\mathcal{O}(dN \log N)$, while the FIP-FL aggregation cost is $\mathcal{O}(|\mathcal{A}^{(t)}|d)$. FIP-FL aggregates only the accepted ciphertexts after verification, while rejected ciphertexts are dropped.

Scalability implication. In the auditor-based PPFL scheme, cost increases with distributional modelling and covariance-related computation. In FIP-FL, verification is linear in the model dimension at the verifier stage. FIP-FL is therefore more scalable for high-dimensional updates because verification avoids cluster-level statistics and covariance estimation.

Here, N is the number of clients, d is the model dimension, B is the batch size, E is the number of local epochs, K is the number of mixture components, D is the auditor-side feature or projection dimension, and $\mathcal{A}^{(t)}$ is the accepted client set in round t.

Overall, this comparison shows that FIP-FL keeps the same client-side learning cost as the auditor-based baseline but replaces auditor-side distributional modelling with a scalar encrypted inner-product audit. For each client, the verifier computes one homomorphic functional inner product with the fixed bootstrap reference direction, giving $\mathcal{O}(Nd)$ score-computation cost. The bootstrap audit then filters scalar scores, contributing $\mathcal{O}(N \log N)$. After verification, only accepted ciphertexts are aggregated, giving $\mathcal{O}(|\mathcal{A}^{(t)}|d)$. Thus, FIP-FL shifts verification from high-dimensional statistical auditing to linear-time encrypted alignment testing, making the verifier stage easier to scale and parallelise across clients.

8.3 Communication Complexity

Table 8.2: Per-round communication cost by directed link.

Link	**Auditor-based [1]**	**FIP-FL**
Client $\rightarrow$ Verifier	$\mathcal{O}(Nd)$	$\mathcal{O}(Nd)$
Verifier $\rightarrow$ Server	$\mathcal{O}(L(Z+N))$	$\mathcal{O}(\lvert\mathcal{A}\rvert d)$
Server $\rightarrow$ Clients	$\mathcal{O}(d)$	$\mathcal{O}(d)$

The first leg of communication, the client-to-verifier payload, is the same in both designs, since both schemes need the encrypted updates to reach the verifier somehow. The savings show up after the audit. FIP-FL never forwards a rejected ciphertext, so the verifier-to-server traffic shrinks with the rejection rate, and the extra auditor-to-server hop that the auditor-based pipeline relies on simply does not exist here.

8.4 Robustness and Engineering Trade-Offs

The detector inside the auditor-based scheme leans on the assumption that the honest-client cluster is both tighter and more populous than the malicious cluster, an assumption that gets noticeably weaker once label skew is in the mix. FIP-FL takes a different tack: it tests, for each individual update, whether the update is aligned with the trusted reference direction and whether its score passes the bootstrap audit. The whole admissibility check stays one-dimensional, and that one-dimensional behaviour is what carries the audit through the heterogeneous cases.

A useful side-effect of this design is that it limits cascade behaviour after a hard round. Only the updates that clear the bootstrap audit ever enter the aggregate, and the bootstrap reference remains fixed across rounds, which prevents a single noisy round from completely re-orienting the audit. The Phase II FIP computations are independent across clients, so they parallelise cleanly, on the 25-core workstation described in Section 6.5 this gave a wall-clock speed-up

of about 13×. The single-reference design also avoids the extra homomorphic inner products that a multi-reference cone would have demanded.

8.5 Scheme-versus-Scheme Accuracy Comparison

Table 8.3: Accuracy on MNIST non-IID under a 50% malicious-client rate. Entries are percentages; baseline values are from [1].

Scheme	**Targeted target-class accuracy ($T_{\mathbf{acc}}$)**	**Untargeted overall accuracy (ACC)**
Krum [4]	71.4	78.3
Auror [7]	32.5	49.9
PEFL [9]	46.4	57.4
Sybils / FoolsGold [15]	89.7	89.5
FL-Trust [8]	37.8	43.4
ShieldFL [16]	90.1	89.4
Auditor-based [1]	96.5	96.3
FIP-FL (this thesis)	**97.69**	88.82

In the table above, FIP-FL records the best target-class accuracy (T_{acc}) of every targeted entry, and it does so while resting on weaker trust assumptions than the auditor-based row. Its untargeted overall accuracy (ACC) is competitive with FoolsGold and ShieldFL but does not quite reach the auditor-based result, and that gap is the clearest empirical place where future work could push further. For the purposes of this thesis the relevant point is the qualitative one: FIP-FL produces the strongest target-class recovery without decrypting any cluster statistics and without bringing in an external auditor.

9 Conclusion and Future Work

9.1 Summary of the Thesis

The argument the thesis has been making is, in one line, that Byzantine-resilient privacy-preserving federated learning does not actually need an external auditor to decrypt cluster-level structure. FIP-FL swaps that trust anchor for a functional inner product taken between each encrypted client update and a fixed bootstrap trusted reference direction. The single scalar that the inner product produces is what feeds the bootstrap audit, whose direction test catches label-flip style reversals.

The proposed approach is implemented through a simple pipeline. Paillier encryption is used at the cryptographic layer, the verifier computes client scores homomorphically, the server aggregates only the accepted encrypted updates, and the bootstrap reference direction remains fixed across rounds. The theoretical analysis lines up with this design, scalar-gradient privacy, a bootstrap audit rejection rule for malicious updates, and a monotone loss-decrease under L-smoothness. Empirically, FIP-FL ends up detecting every malicious-client configuration in the completed grid, lifts MNIST non-IID target-class accuracy from roughly 7% in the undefended baseline to 97.69% at a 50% malicious-client rate, and replaces the auditor's $\mathcal{O}(d^3)$ covariance-inversion step with a verification stage that parallelises to a 13× wall-clock speed-up in Phase II.

9.2 Limitations and Future Work

The main limitations of this thesis arise from the experimental setting. FIP-FL is evaluated using softmax logistic regression models, parameter dimensions in

the low-thousand range, a fixed client population, and a single attack type in each run.

Extending the method to deep networks will likely require packed homomorphic encryption schemes such as BFV or CKKS [12]. A more realistic setting should also consider client churn, evolving data distributions, adaptive adversaries, and combined attack strategies such as label flipping together with update scaling.

Another limitation concerns the initialization of the reference direction. In the current design, this direction is derived from a small public dataset. Although this works well in the present experiments, it assumes the availability of such public data.

A stronger approach for the first round would be to estimate the reference direction from robust aggregate statistics computed over the clients themselves, while still avoiding decryption of individual gradients. Beyond this, FIP-FL should be evaluated on real laptop, mobile, and edge devices to better understand the trade-off among encryption overhead, communication cost, and model utility in practical privacy-preserving federated learning settings.

Bibliography

[1] Abbas Yazdinejad, Ali Dehghantanha, Hadis Karimipour, Gautam Srivastava, and Reza M. Parizi. A robust privacy-preserving federated learning model against model poisoning attacks. *IEEE Transactions on Information Forensics and Security*, 19:6693–6708, 2024.

[2] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 54, pages 1273–1282, 2017.

[3] Ligeng Zhu, Zhijian Liu, and Song Han. Deep leakage from gradients. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[4] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine-tolerant gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[5] Dong Yin, Yudong Chen, Kannan Ramchandran, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 5650–5659, 2018.

[6] El Mahdi El Mhamdi, Rachid Guerraoui, and Sébastien Rouault. The hidden vulnerability of distributed learning in Byzantium. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 3521–3530, 2018.

[7] Shiqi Shen, Shruti Tople, and Prateek Saxena. Auror: Defending against poisoning attacks in collaborative deep learning systems. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*, pages 508–519, 2016.

[8] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium (NDSS)*, 2021.

[9] Xiaoyuan Liu, Hongwei Li, Guowen Xu, Zongqi Chen, Xiaoming Huang, and Rongxing Lu. Privacy-enhanced federated learning against poisoning adversaries. *IEEE Transactions on Information Forensics and Security*, 16: 4574–4588, 2021.

[10] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, pages 1175–1191, 2017.

[11] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *Advances in Cryptology – EUROCRYPT '99*, pages 223–238. Springer, 1999.

[12] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, pages 409–437. Springer, 2017.

[13] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. Analyzing federated learning through an adversarial lens. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 634–643, 2019.

[14] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. How to backdoor federated learning. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 2938–2948, 2020.

[15] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. The limitations of federated learning in Sybil settings. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, pages 301–316, 2020.

[16] Zhuoran Ma, Jianfeng Ma, Yinbin Miao, Yuan Li, and Robert H. Deng. ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning. *IEEE Transactions on Information Forensics and Security*, 17:1639–1654, 2022.

[17] Michel Abdalla, Florian Bourse, Angelo De Caro, and David Pointcheval. Simple functional encryption schemes for inner products. In *Public-Key Cryptography – PKC 2015*, volume 9020 of *Lecture Notes in Computer Science*, pages 733–751. Springer, 2015. doi: 10.1007/978-3-662-46447-2_33.